PERTEMUAN 10

# Dasar-dasar Deteksi Objek

Pengolahan Citra dan Visi Komputer · RTI245007

# Tujuan Pembelajaran & Outline

## Tujuan Pembelajaran

1. Memahami konsep dan prinsip dasar deteksi objek.
2. Menjelaskan aspek pemindaian dalam deteksi objek.
3. Mengenali metode dasar deteksi objek menggunakan OpenCV.

## Outline Materi

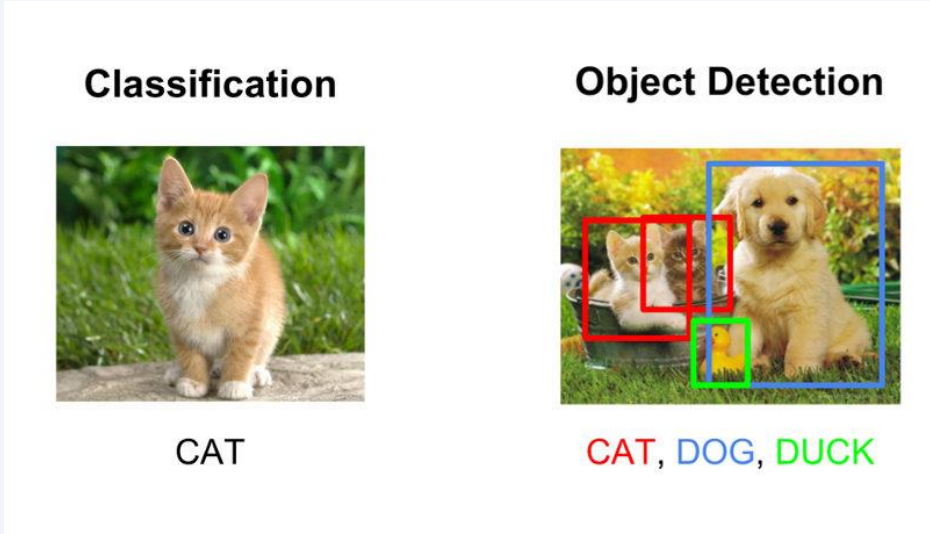
1. Dasar-dasar Deteksi Objek
2. Konsep dan Prinsip Deteksi Objek
3. Aspek Pemindaian pada Deteksi Objek
4. Face Detection

# Deteksi Objek (Object Detection)

Teknik **computer vision** untuk **mengidentifikasi dan menemukan objek** dalam sebuah gambar atau video.

Menggambar **kotak pembatas (bounding box)** di sekitar objek yang terdeteksi, sehingga lokasi objek pada gambar/video dapat diketahui.

**Tujuan:** mendeteksi semua instance objek dari citra tersebut.




**Klasifikasi**: satu label untuk seluruh gambar.
**Deteksi objek**: label sekaligus lokasi tiap objek.

# Tujuan Deteksi Objek

Deteksi objek adalah **tahap pertama**: hasilnya (objek dan lokasinya) memungkinkan diperolehnya informasi lebih lanjut, antara lain:



### Mengenali contoh spesifik

Mis. mengidentifikasi subjek dari wajah yang terdeteksi.



### Melacak objek pada urutan gambar

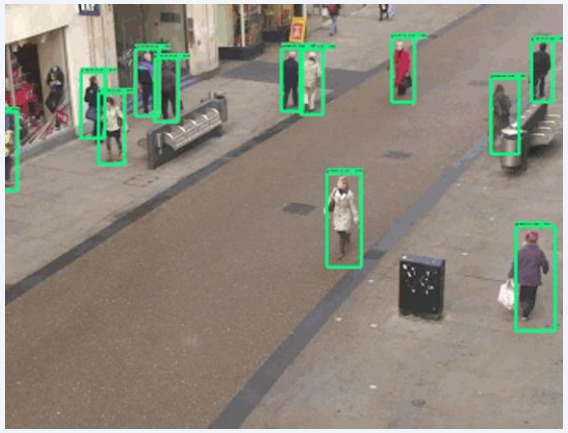
Mis. melacak wajah dalam video.



### Mengekstrak informasi lebih lanjut

Mis. menentukan jenis kelamin subjek.

# Contoh Implementasi Object Detection



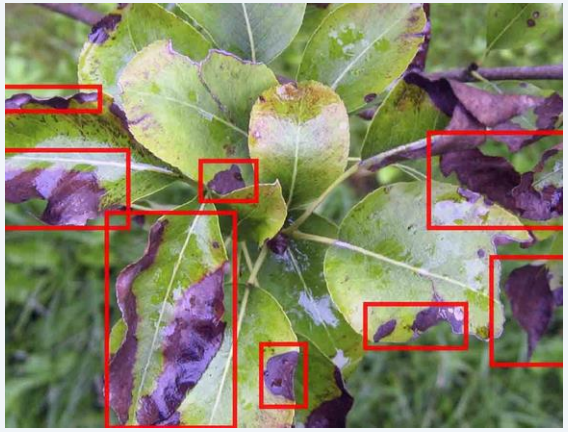
### Video surveillance (CCTV)

Memantau area dan mengenali orang atau kendaraan pada rekaman kamera.



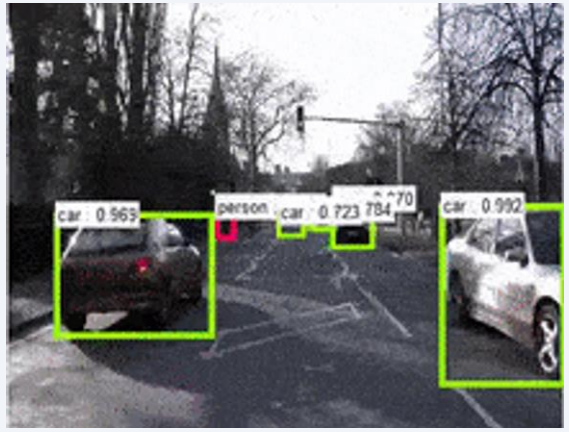

### Crowd counting (penghitungan kerumunan)

Menghitung jumlah orang pada area yang ramai.



### Anomaly detection

Menemukan bagian yang menyimpang dari kondisi normal, mis. daun yang terserang penyakit.




### Self-driving cars

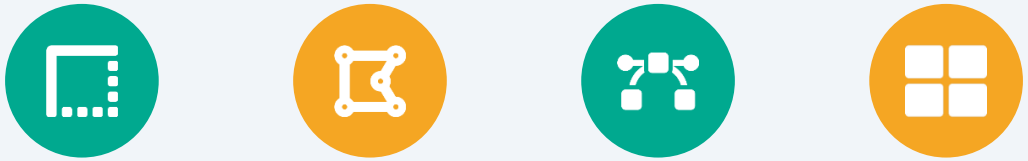
Mendeteksi kendaraan dan pejalan kaki di jalan.

# Konsep dan Prinsip Dasar Deteksi Objek

## Konsep dasar citra dan objek

Objek dikenali dari perbedaan **intensitas, warna, dan tekstur** terhadap latar belakang.

## Elemen penting:



- Edges
- Corners
- Contours
- Texture patterns

## Prinsip dasar deteksi objek: langkah utama

1. **Ekstraksi fitur awal**
2. **Deskripsi fitur**
3. **Matching / Klasifikasi**
4. **Lokalisasi (bounding box)**

# Metode Deteksi Dasar

1

## Template Matching

Mencari lokasi gambar template pada gambar yang lebih besar.

2

## Edge Detection

Mendeteksi garis tepi yang membatasi dua wilayah dengan kecerahan berbeda.

**Sobel · Laplacian · Canny**

3

## Corner Detection

Mencari titik sudut, yaitu persimpangan dua sisi.

**Harris · Shi-Tomasi**

4

## Grid Detection

Mendeteksi garis-garis grid dengan Hough Transform.

5

## Contour Detection

Menemukan kurva tertutup yang mewakili bentuk objek.

# 1. Template Matching

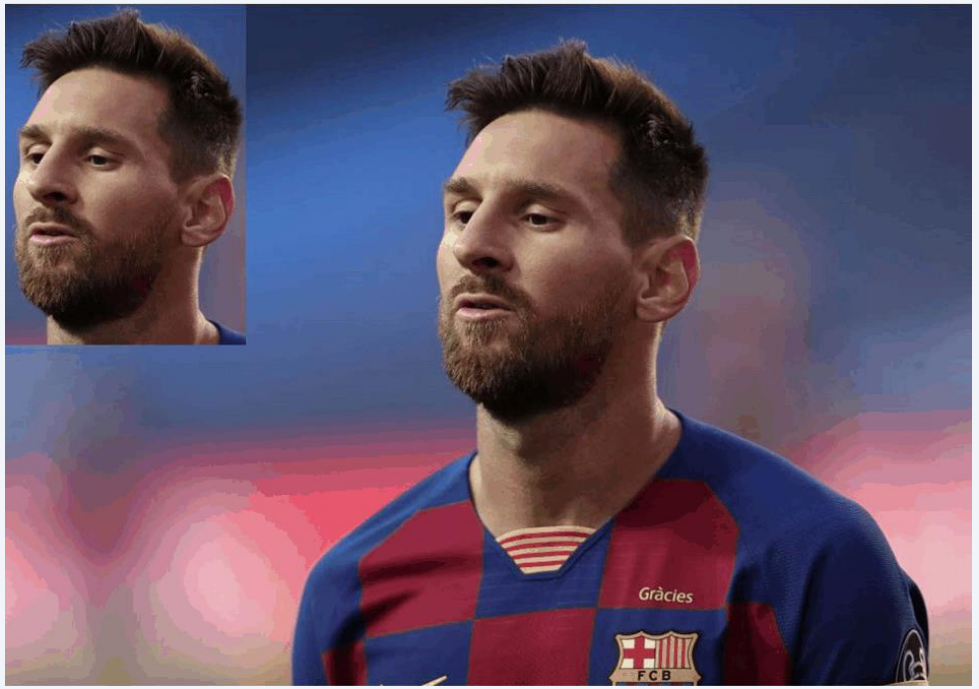
Metode untuk **mencari dan menemukan lokasi gambar template** pada gambar yang lebih besar. Diperlukan dua gambar:

I

**Source image (I)**
Gambar masukan yang diharapkan cocok dengan template.

T

**Template image (T)**
Potongan objek yang akan dicari pada source image.




*Contoh: potongan wajah (template, kiri atas) dicari pada gambar sumber*

# 1. Template Matching di OpenCV

OpenCV mengimplementasikan template matching dengan fungsi `matchTemplate()`

1. method=TM_SQDIFF

$$R(x,y) = \sum_{x',y'} (T(x',y') - I(x+x', y+y'))^2$$

2. method=TM_SQDIFF_NORMED

$$R(x,y) = \frac{\sum_{x',y'} (T(x',y') - I(x+x', y+y'))^2}{\sqrt{\sum_{x',y'} T(x',y')^2 \cdot \sum_{x',y'} I(x+x', y+y')^2}}$$

3. method=TM_CCORR

$$R(x,y) = \sum_{x',y'} (T(x',y') \cdot I(x+x', y+y'))$$

4. method=TM_CCORR_NORMED

$$R(x,y) = \frac{\sum_{x',y'} (T(x',y') \cdot I(x+x', y+y'))}{\sqrt{\sum_{x',y'} T(x',y')^2 \cdot \sum_{x',y'} I(x+x', y+y')^2}}$$

5. method=TM_CCOEFF

$$R(x,y) = \sum_{x',y'} (T'(x',y') \cdot I'(x+x', y+y'))$$

6. method=TM_CCOEFF_NORMED

$$R(x,y) = \frac{\sum_{x',y'} (T'(x',y') \cdot I'(x+x', y+y'))}{\sqrt{\sum_{x',y'} T'(x',y')^2 \cdot \sum_{x',y'} I'(x+x', y+y')^2}}$$

**Dokumentasi template matching pada OpenCV:**

docs.opencv.org/3.4/de/da9/tutorial_template_matching.html

CONTOH PERHITUNGAN

# Di satu posisi, TM_SQDIFF menghasilkan R = 18

**Template T**

| | |
|---|---|
| 10 | 15 |
| 20 | 25 |

**Potongan gambar I**

| | |
|---|---|
| 12 | 18 |
| 22 | 26 |

Template diletakkan di atas gambar.

Piksel di posisi yang sama dibandingkan satu per satu.

**Selisih tiap piksel, lalu dikuadratkan**

| Posisi (x′, y′) | T | I | T − I | (T − I)² |
|---|---|---|---|---|
| (0, 0) | 10 | 12 | −2 | **4** |
| (0, 1) | 15 | 18 | −3 | **9** |
| (1, 0) | 20 | 22 | −2 | **4** |
| (1, 1) | 25 | 26 | −1 | **1** |

**R(x, y) = 4 + 9 + 4 + 1 = 18**

CONTOH PERHITUNGAN

# Mencari patch 2×2 di dalam gambar 6×6

**Gambar (img), 6×6**

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| 0 | 1 | 3 | 2 | 5 | 8 | 9 |
| 1 | 2 | 4 | 6 | 8 | 9 | 9 |
| 2 | 7 | 6 | 4 | 8 | 9 | 6 |
| 3 | 1 | 4 | 6 | 7 | 3 | 9 |
| 4 | 6 | 4 | 7 | 9 | 6 | 4 |
| 5 | 8 | 6 | 4 | 6 | 8 | 9 |

**Template (patch)**

| | |
|---|---|
| 6 | 4 |
| 4 | 6 |

**Langkah TM_SQDIFF**

- Letakkan patch di sudut kiri atas gambar, posisi **(0, 0)**
- Hitung $\Sigma(T - I)^2$ untuk 4 piksel yang tertutup
- Geser satu piksel ke kanan, ulangi sampai ujung baris, lalu turun satu baris
- Hasilnya peta **(6 – 2 + 1) × (6 – 2 + 1) = 5 × 5**
- **Skor terkecil = posisi paling mirip**

CONTOH PERHITUNGAN

# Posisi pertama (0, 0) menghasilkan R = 34

**Gambar, patch di (0, 0)**

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| 0 | 1 | 3 | 2 | 5 | 8 | 9 |
| 1 | 2 | 4 | 6 | 8 | 9 | 9 |
| 2 | 7 | 6 | 4 | 8 | 9 | 6 |
| 3 | 1 | 4 | 6 | 7 | 3 | 9 |
| 4 | 6 | 4 | 7 | 9 | 6 | 4 |
| 5 | 8 | 6 | 4 | 6 | 8 | 9 |

| T − I | | Selisih | | Dikuadratkan | |
|---|---|---|---|---|---|
| 6−1 | 4−3 | 5 | 1 | 25 | 1 |
| 4−2 | 6−4 | 2 | 2 | 4 | 4 |

**25 + 1 + 4 + 4 = 34**

Keempat kuadrat dijumlahkan menjadi satu skor, lalu ditulis di sel (0, 0) peta hasil.

**Peta hasil R**

| | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 0 | 34 | | | | |
| 1 | | | | | |
| 2 | | | | | |
| 3 | | | | | |
| 4 | | | | | |

CONTOH PERHITUNGAN

# Digeser ke 25 posisi, hasilnya matrix 5×5

**Peta hasil R (TM_SQDIFF)**

| | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 0 | 34 | 13 | 25 | 42 | 63 |
| 1 | 25 | 16 | 20 | 54 | 59 |
| 2 | 18 | 0 | 25 | 47 | 23 |
| 3 | 33 | 9 | 27 | 27 | 42 |
| 4 | 16 | 21 | 26 | 21 | 25 |

- Setiap sel = skor R saat sudut kiri atas patch berada di posisi (x, y)
- Contoh: sel (0, 0) = **34**, hasil dari slide sebelumnya
- Peta lebih kecil dari gambar, karena patch tidak boleh keluar dari tepi

# Skor 0 di (x = 1, y = 2) menandai lokasi patch

**Peta hasil R**

| | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 0 | 34 | 13 | 25 | 42 | 63 |
| 1 | 25 | 16 | 20 | 54 | 59 |
| 2 | 18 | **0** | 25 | 47 | 23 |
| 3 | 33 | 9 | 27 | 27 | 42 |
| 4 | 16 | 21 | 26 | 21 | 25 |

**Gambar, cocok di sudut kiri atas**

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| 0 | 1 | 3 | 2 | 5 | 8 | 9 |
| 1 | 2 | 4 | 6 | 8 | 9 | 9 |
| 2 | 7 | **6** | 4 | 8 | 9 | 6 |
| 3 | 1 | 4 | 6 | 7 | 3 | 9 |
| 4 | 6 | 4 | 7 | 9 | 6 | 4 |
| 5 | 8 | 6 | 4 | 6 | 8 | 9 |

## Membaca hasil

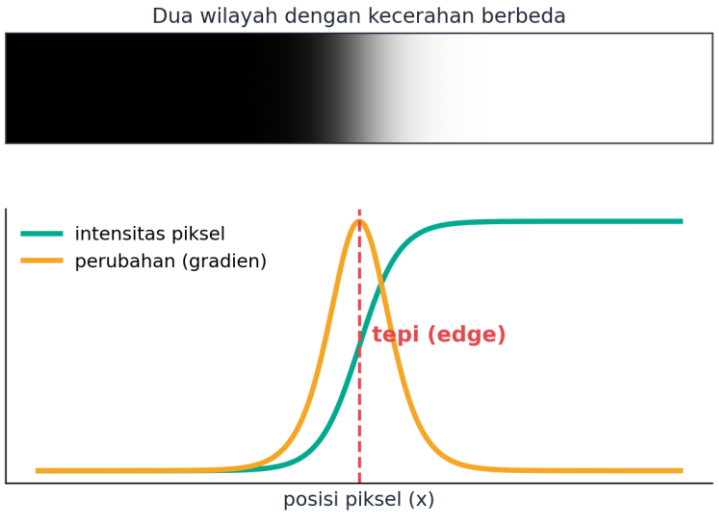
- Nilai 0 berarti patch **identik** dengan bagian gambar
- Koordinat yang didapat adalah **sudut kiri atas** area yang cocok

```
minV, maxV, minLoc, maxLoc =
  cv2.minMaxLoc(R)
# minLoc = (1, 2)
```

# 2. Edge Detection

Operasi untuk mendeteksi **garis tepi (edges)** yang membatasi dua wilayah citra dengan tingkat kecerahan berbeda.

Suatu titik (x, y) adalah **tepi** bila memiliki perbedaan yang tinggi dengan tetangganya.




### Sobel Edge Detection

Menghitung gradien intensitas citra pada setiap piksel.

### Laplacian Edge Detection

Sensitif terhadap noise, sehingga citra dihaluskan dulu dengan filter Gaussian.

### Canny Edge Detection

Bekerja dalam 4 tahap: Gaussian, gradien, non-maximum suppression, hysteresis.

# 2. Edge Detection: Sobel & Laplacian

## Sobel Edge Detection

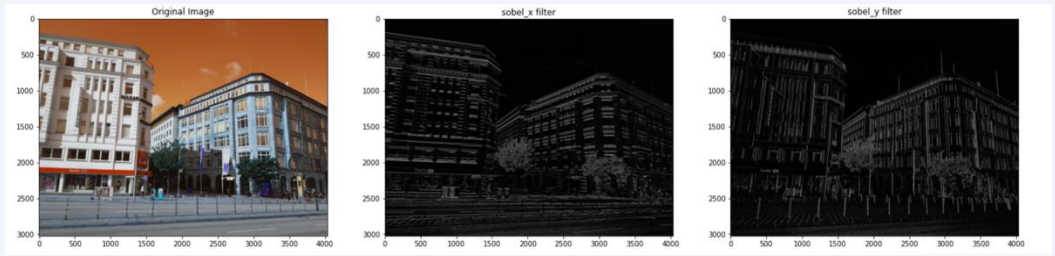
Filter Sobel bekerja dengan menghitung **gradien intensitas** citra pada setiap piksel.

| | | |
|---|---|---|
| -1 | 0 | +1 |
| -2 | 0 | +2 |
| -1 | 0 | +1 |

Gx

| | | |
|---|---|---|
| +1 | +2 | +1 |
| 0 | 0 | 0 |
| -1 | -2 | -1 |

Gy



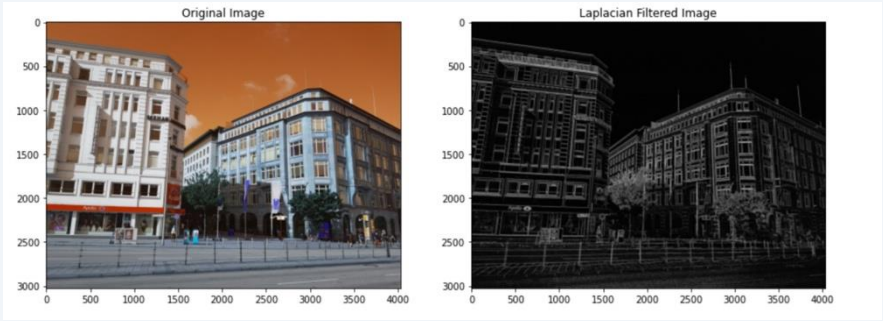

*Kernel Gx dan Gy, serta hasil filter Sobel*

## Laplacian Edge Detection

Sangat **sensitif terhadap noise**, sehingga citra umumnya dihaluskan dulu dengan filter Gaussian sebelum Laplacian.

| | | |
|---|---|---|
| 0 | −1 | 0 |
| −1 | 4 | −1 |
| 0 | −1 | 0 |

| | | |
|---|---|---|
| −1 | −1 | −1 |
| −1 | 8 | −1 |
| −1 | −1 | −1 |

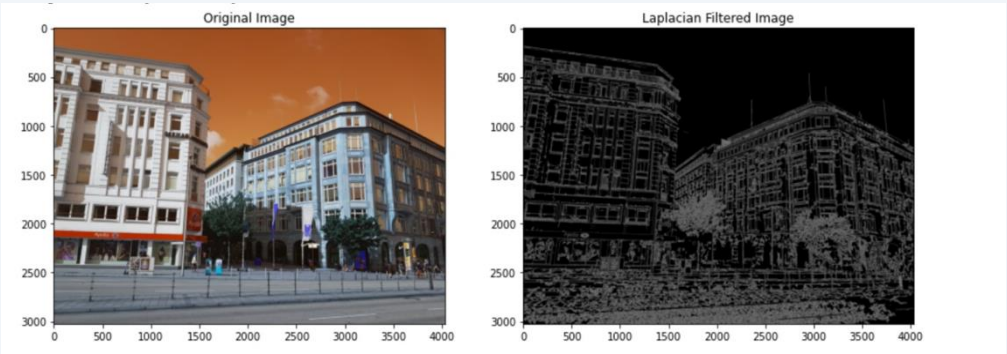



*Kernel Laplacian dan hasil pada citra*

# 2. Edge Detection: Canny

## Canny bekerja dengan 4 tahap:

1. Menghaluskan gambar dengan **filter Gaussian** untuk mengurangi noise
2. Menghitung **gradien** menggunakan salah satu operator gradien Sobel
3. Mengekstraksi titik tepi: **Non-maximum suppression**
4. Menghubungkan dan thresholding: **Hysteresis**



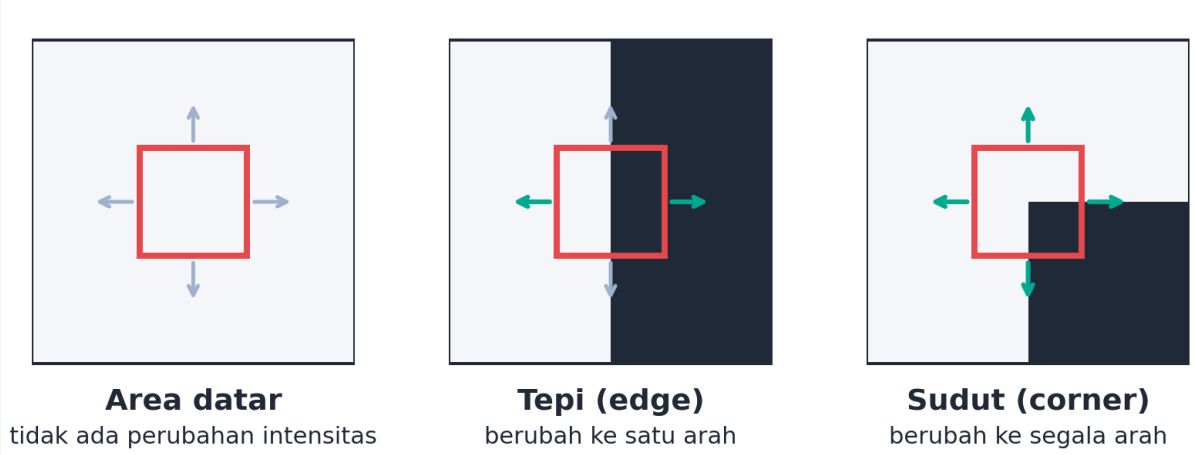

*Citra asli dan hasil Canny Edge Detection*

# 3. Corner Detection

**Sudut** dapat diartikan sebagai persimpangan dua sisi.

Titik sudut tidak bisa didefinisikan dari piksel tunggal, karena di sana hanya terdapat satu gradien di setiap titik.

**Gradien** adalah arah perubahan intensitas kecerahan dalam suatu citra.




**Metode:** Harris | Shi-Tomasi

Sudut = titik yang intensitasnya **berubah ke segala arah** ketika window digeser.

# 3. Corner Detection: Harris

Didasarkan pada **variasi intensitas** sinyal; variasi yang besar menunjukkan adanya sudut pada citra.

Cara kerja: mencari perbedaan intensitas untuk perpindahan (u, v) ke segala arah, dengan persamaan:

$$E(u,v) = \sum_{x,y} \underbrace{w(x,y)}_{\text{window function}} \underbrace{[I(x+u,y+v)}_{\text{shifted intensity}} - \underbrace{I(x,y)}_{\text{intensity}}]^2$$

## Pada OpenCV:

```
cv2.cornerHarris(src, dest, blockSize, kSize, freeParameter, borderType)
```

**src**
citra input

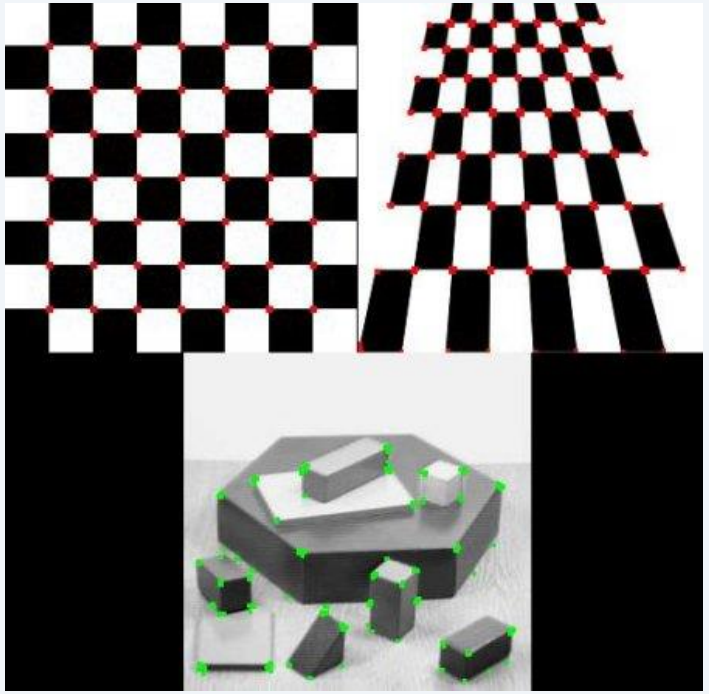
**dest**
citra untuk menyimpan hasil Harris detector

**blockSize**
ukuran ketetanggaan

**ksize**
kernel size untuk operator Sobel

**freeParameter**
parameter bebas untuk Harris detector

**borderType**
metode pixel extrapolation

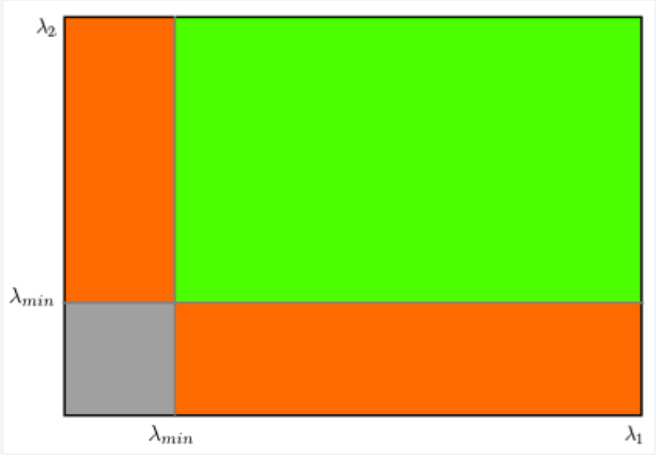


*Contoh citra uji deteksi sudut*

# 3. Corner Detection: Shi-Tomasi

Dasar: sudut dapat dideteksi dengan mencari **perubahan signifikan ke segala arah**.

$$R = \min(\lambda_1, \lambda_2)$$




*Hijau: sudut · oranye: tepi · abu-abu: datar*

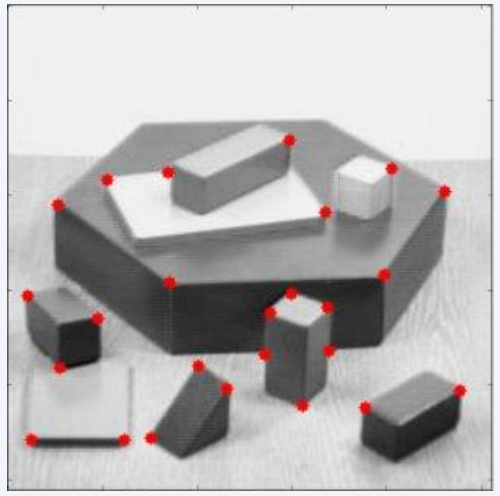
**Pada OpenCV:**

```
cv.goodFeaturesToTrack()
```

Fungsi ini menemukan **N sudut terkuat** pada gambar dengan metode Shi-Tomasi.

**Parameter:**

- **a** — jumlah sudut yang ingin ditemukan
- **b** — tingkat kualitas, bernilai antara 0–1
- **c** — jarak euclidean minimum antar sudut yang terdeteksi



*Hasil: sudut terkuat ditandai titik merah*

# Contoh corner detection: citra 8×8 dengan satu sudut

**Citra I (0 = gelap, 10 = terang)**

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 3 | 0 | 0 | 0 | 10 | 10 | 10 | 10 | 10 |
| 4 | 0 | 0 | 0 | 10 | 10 | 10 | 10 | 10 |
| 5 | 0 | 0 | 0 | 10 | 10 | 10 | 10 | 10 |
| 6 | 0 | 0 | 0 | 10 | 10 | 10 | 10 | 10 |
| 7 | 0 | 0 | 0 | 10 | 10 | 10 | 10 | 10 |

**Window sudut**
pusat (3, 3)

**Window tepi**
pusat (3, 6)

**Window datar**
pusat (6, 6)

### Langkah perhitungan

1. Hitung gradien **Ix** dan **Iy** tiap piksel
2. Di tiap window 3×3, jumlahkan $Ix^2$, $Iy^2$, dan $Ix \cdot Iy$ menjadi matriks **M**
3. Harris: $\mathbf{R = det(M) - k \cdot (trace\ M)^2}$, k = 0,04
4. Shi-Tomasi: $\mathbf{R = min(\lambda 1, \lambda 2)}$

# Gradien hanya muncul di batas gelap–terang

**Ix, perubahan ke kanan**

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 3 | 0 | 0 | **10** | 0 | 0 | 0 | 0 | 0 |
| 4 | 0 | 0 | **10** | 0 | 0 | 0 | 0 | 0 |
| 5 | 0 | 0 | **10** | 0 | 0 | 0 | 0 | 0 |
| 6 | 0 | 0 | **10** | 0 | 0 | 0 | 0 | 0 |
| 7 | 0 | 0 | **10** | 0 | 0 | 0 | 0 | 0 |

**Iy, perubahan ke bawah**

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 2 | 0 | 0 | 0 | **10** | **10** | **10** | **10** | **10** |
| 3 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 4 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 5 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 6 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 7 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

$$I_x(x, y) = I(x+1, y) - I(x, y) \qquad I_y(x, y) = I(x, y+1) - I(x, y)$$

- **Ix** bernilai 10 di sepanjang tepi tegak
- **Iy** bernilai 10 di sepanjang tepi datar
- Di area rata, keduanya 0
- OpenCV memakai Sobel (ksize); idenya sama

CONTOH PERHITUNGAN

# Window sudut di (3, 3) menghasilkan R Harris = 33.600

**Ix**

| | | |
|---|---|---|
| 0 | 0 | 0 |
| **10** | 0 | 0 |
| **10** | 0 | 0 |

**Iy**

| | | |
|---|---|---|
| 0 | **10** | **10** |
| 0 | 0 | 0 |
| 0 | 0 | 0 |

**$Ix^2$**

| | | |
|---|---|---|
| 0 | 0 | 0 |
| **100** | 0 | 0 |
| **100** | 0 | 0 |

**Σ = 200**

**$Iy^2$**

| | | |
|---|---|---|
| 0 | **100** | **100** |
| 0 | 0 | 0 |
| 0 | 0 | 0 |

**Σ = 200**

**Ix · Iy**

| | | |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 0 | 0 |
| 0 | 0 | 0 |

**Σ = 0**

**Matriks M**

| | |
|---|---|
| 200 | 0 |
| 0 | 200 |

Diagonal: $\Sigma Ix^2$ dan $\Sigma Iy^2$
Sisi lain: $\Sigma Ix \cdot Iy$

$\det(M) = 200 \cdot 200 - 0^2 = 40.000$ $\quad$ $\text{trace}(M) = 200 + 200 = 400$

**$R = 40.000 - 0{,}04 \cdot 400^2 = 40.000 - 6.400 =$ 33.600**

Shi-Tomasi: $\lambda 1 = \lambda 2 = 200$, jadi $R = \min(200, 200) =$ **200**

CONTOH PERHITUNGAN

# Hanya window sudut yang punya dua nilai λ besar

| Window | Matriks M | λ1, λ2 | R Harris | R Shi-Tomasi | Hasil |
|---|---|---|---|---|---|
| **Datar (6, 6)** | [0 0 ; 0 0] | 0, 0 | 0 | 0 | Datar |
| **Tepi (3, 6)** | [300 0 ; 0 0] | 300, 0 | −3.600 | 0 | Tepi |
| **Sudut (3, 3)** | [200 0 ; 0 200] | 200, 200 | **33.600** | **200** | **Sudut** |

**Harris**

$R > 0 \rightarrow$ sudut, $R < 0 \rightarrow$ tepi, $R \approx 0 \rightarrow$ datar

Tanda R sudah cukup untuk membedakan ketiganya

**Shi-Tomasi**

Sudut jika λ terkecil pun di atas ambang

Tepi punya satu λ = 0, jadi langsung tersaring

# 4. Grid Detection

## Dilakukan dengan metode Hough Transform. Tahapan:

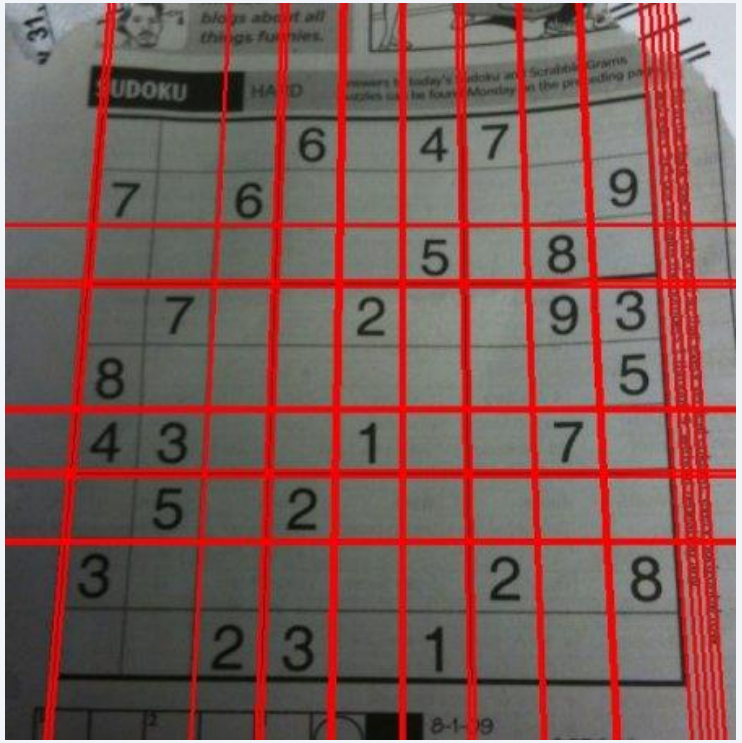
1. **Terapkan Canny Edge Detection**

2. **Dilasi gambar tepi**
   Canny dapat menemukan kedua tepi pemisah grid sebagai tepi berbeda; dilasi menggabungkannya kembali.

3. **Erosi**
   Setelah dilasi, garis border menjadi tebal sehingga Hough akan mendeteksi banyak garis.

4. **Terapkan HoughLines**
   Menggunakan fungsi cv2.HoughLines() pada OpenCV.

5. **Gabungkan garis yang mirip (similar)**




*Hasil: garis grid terdeteksi (merah)*

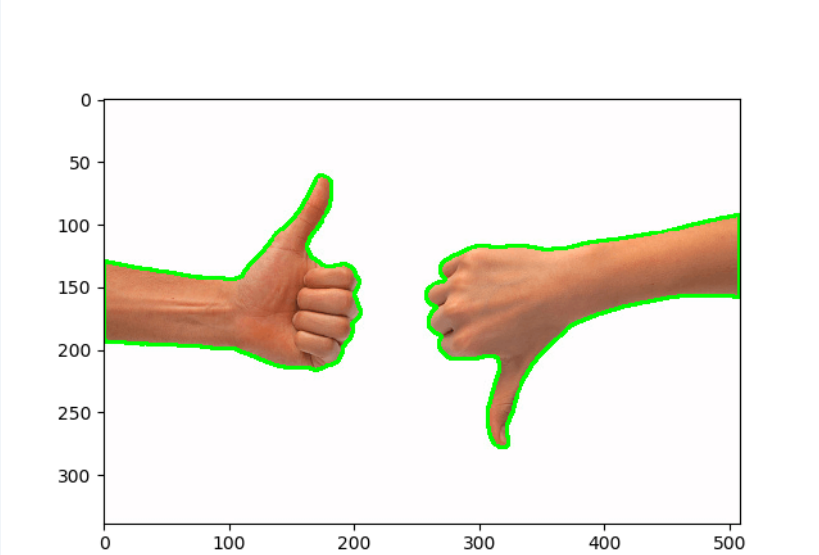
# 5. Contour Detection

**Kontur** adalah kurva tertutup yang menghubungkan semua titik kontinu dengan warna atau intensitas yang sama; kontur mewakili **bentuk objek** dalam gambar.

## Tahapan:

1. Ubah citra input menjadi **citra biner** (thresholding atau Canny edge detection).
2. Temukan kontur dengan fungsi **findContours()** pada OpenCV.
3. Gambarkan kontur pada citra input dengan fungsi **drawContours()**.

**Parameter findContours():**
1) citra yang dianalisis konturnya 2) metode retrieval 3) metode approximation bentuk (shape)




*Kontur (hijau) mengikuti bentuk objek*

**Dokumentasi:**
docs.opencv.org/master/d4/d73/tutorial_py_contours_begin.html

# Aspek Pemindaian dalam Deteksi Objek

Pemindaian dilakukan untuk **menelusuri seluruh area citra**, dapat per baris, blok, atau region of interest (ROI).

### 1. Pemindaian spasial

*(spatial scanning)*

Per baris atau blok untuk mendeteksi perubahan nilai intensitas.

**Sliding window**: template dipindai ke seluruh area citra dengan langkah tertentu.

**Pyramid scaling**: citra diubah ukurannya untuk mendeteksi objek berbagai ukuran.

### 2. Pemindaian multiskala & orientasi

Pada SIFT dan ORB, objek terdeteksi meski ukuran dan rotasi berubah.

Setiap skala menghasilkan keypoints berbeda, lalu dipilih yang paling signifikan.

### 3. Pemindaian berbasis region

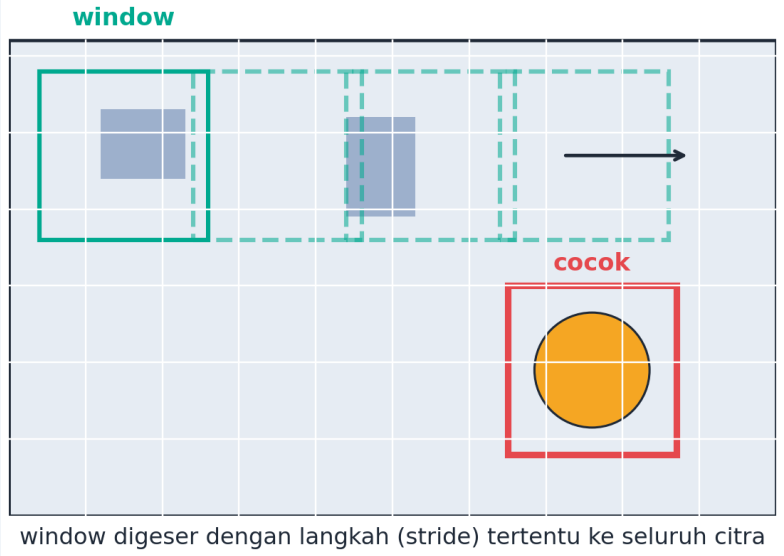
*(region proposal)*

Dipakai pada deteksi modern seperti R-CNN, YOLO, dan SSD.

Citra dibagi menjadi beberapa ROI untuk mempercepat proses deteksi.

# Sliding Window & Multi-Scale Scanning

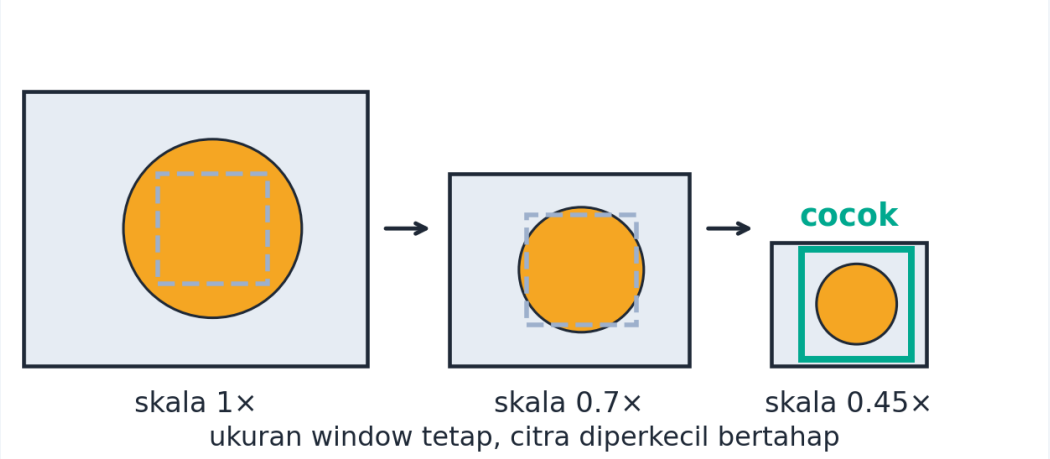
## Sliding Window Technique




Jendela geser dipakai untuk **mencari kecocokan pola** di seluruh area citra.

Dipakai pada deteksi wajah, plat nomor, atau objek kecil.

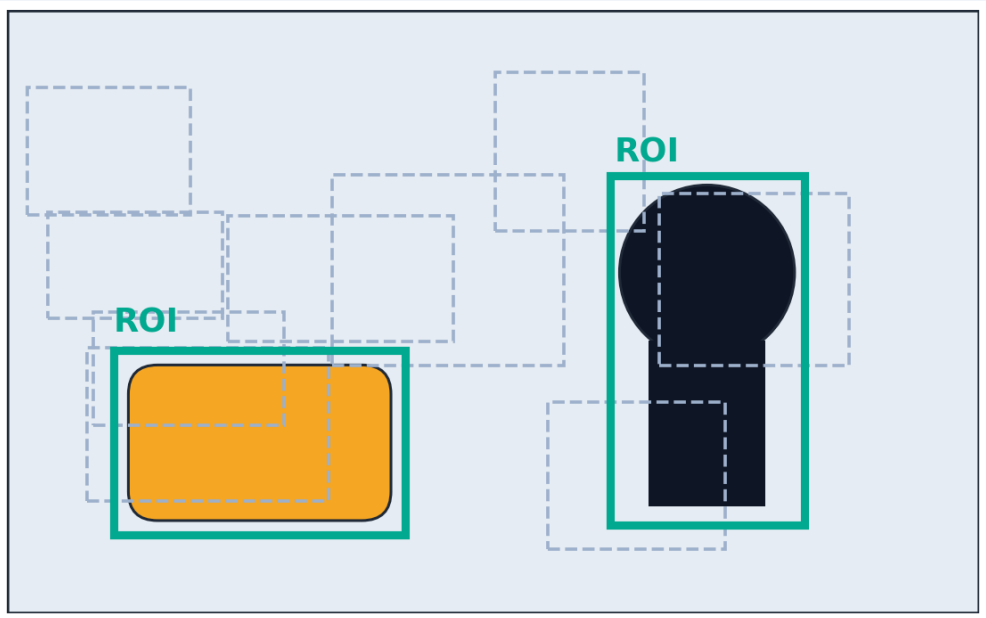
## Multi-Scale Scanning




Citra **diubah ukurannya** agar objek dari berbagai ukuran dapat terdeteksi.

Dipakai dalam SIFT dan ORB.

# Region-Based Scanning (ROI)




usulan region (garis putus) disaring menjadi ROI yang berisi objek

Metode modern membagi citra ke **area potensial (region of interest)** agar proses deteksi lebih efisien.

**Contoh metode:**

- R-CNN
- YOLO
- SSD

**Integrasi fitur dan pemindaian**
Kombinasi fitur (edge, corner, contour) dan pemindaian menghasilkan sistem deteksi yang efisien dan akurat.

# Feature Matching

**Citra sumber & citra template**

**Ekstraksi fitur penting (dasar: corner, edge, contour detection)**

**Hitung distance untuk mencari kecocokan sumber–template**

Jika objek pada citra template **tidak berukuran sama** dengan citra sumber, deteksi tetap bisa dilakukan dengan baik.

**ORB**

Brute-Force Matching menggunakan **ORB Descriptor** (Oriented FAST and Rotated BRIEF)

**SIFT**

Brute-Force Matching menggunakan **SIFT Descriptor** dan Test Ratio (Scale-Invariant Feature Transform)

**Referensi:**

pyimagesearch.com/2015/01/26/multi-scale-template-matching-using-python-opencv

laptrinhx.com/opencv-template-matching-cv2-matchtemplate-2183411893

# ORB (Oriented FAST and Rotated BRIEF)

## Sekilas ORB

Berasal dari **OpenCV Labs**; diajukan Ethan Rublee, Vincent Rabaud, Kurt Konolige, dan Gary R. Bradski dalam artikel "ORB: An efficient alternative to SIFT or SURF" (2011).

Alternatif pengganti **SIFT dan SURF** dalam hal biaya komputasi, kinerja matching, dan terutama **paten**.

**SIFT & SURF**
dipatenkan, harus membayar untuk memakainya

**ORB**
tidak perlu membayar untuk penggunaannya

Cara kerja: perpaduan detektor keypoint **FAST** dan deskriptor **BRIEF** dengan banyak modifikasi untuk meningkatkan kinerja.

1. **FAST menemukan keypoint**
   Langkah awal untuk mendapatkan titik-titik kunci pada citra.

2. **Harris Corner Detection**
   Dipakai untuk mencari N-point tertinggi dari keypoint FAST.

3. **Modifikasi orientasi**
   FAST tidak menghitung orientasi, sehingga dimodifikasi agar algoritma invariant terhadap orientasi.

# ORB: Contoh Hasil Matching

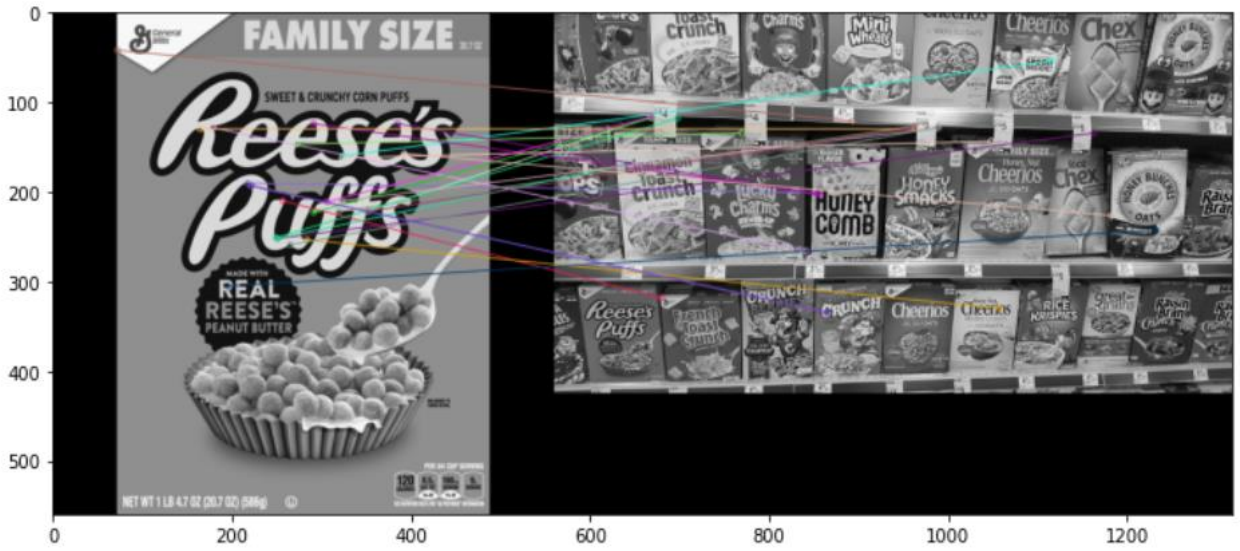



*Citra template*




*Citra sumber (rak)*

```
matches = sorted(matches,key=lambda x:x.distance)
reeses_match = cv.drawMatches(reeses,kp1,cereals,kp2,matches[:25],None,flags=2)
display(reeses_match)
```




Hasil **Brute-Force Matching** dengan ORB descriptor: garis menghubungkan keypoint pada template dengan keypoint pada citra sumber.

# SIFT (Scale-Invariant Feature Transform)

### Rotation-invariant

Harris corner dan metode sejenis tetap bisa mendeteksi corner walaupun citra dirotasi, karena **corner tetap corner**.

### Tidak scale-invariant

Pada scaling, **corner bisa jadi bukan lagi corner** jika citra diskalakan. Artinya Harris corner bukan algoritma scale-invariant.

### SIFT (D. Lowe, 2004)

University of British Columbia, artikel “Distinctive Image Features from Scale-Invariant Keypoints”: mengekstrak keypoint dan menghitung deskriptornya.



**Citra asli**
**sudut di dalam window**

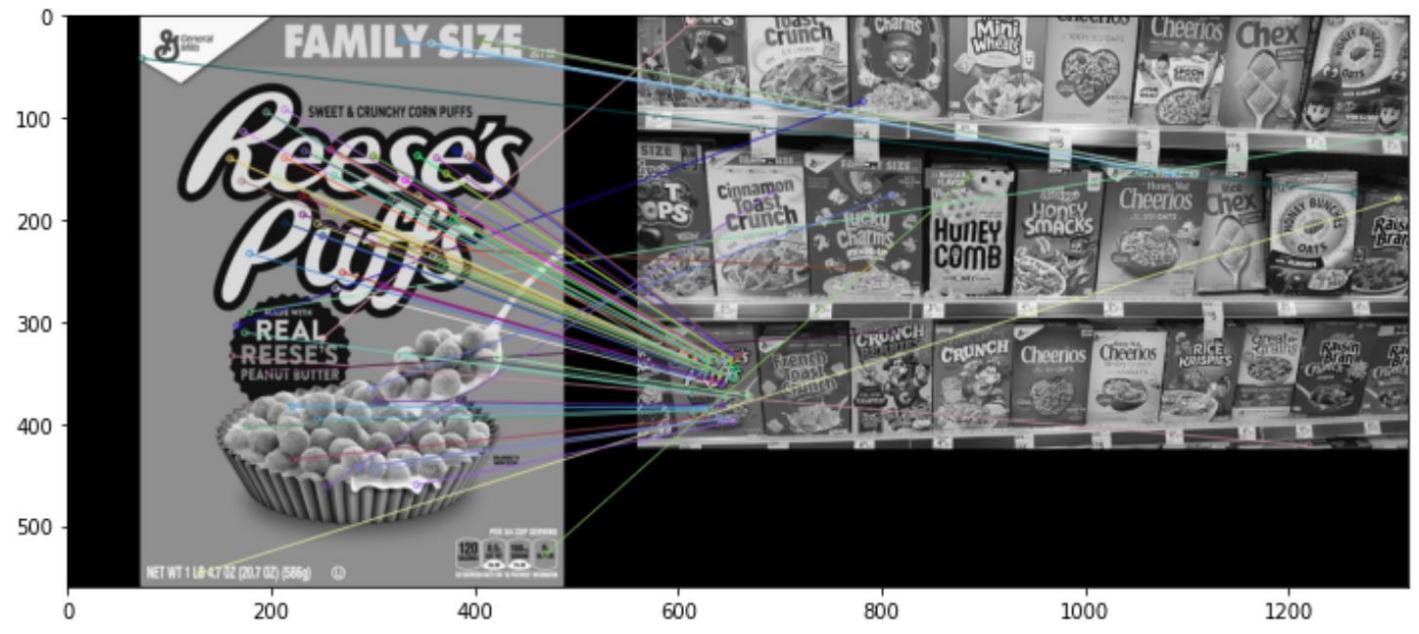
**Dirotasi**
**tetap tampak sebagai sudut**

**Diskalakan (zoom)**
**window hanya melihat tepi**

### Catatan penggunaan

Gunakan **OpenCV versi 4.4.0.44 ke atas**. Lakukan instalasi lewat **pip** sebelum menjalankan kode SIFT.

# SIFT: Contoh Hasil & Perbandingan dengan ORB




Hasil **Brute-Force Matching** dengan SIFT descriptor dan **test ratio**.

| | ORB | SIFT |
|---|---|---|
| **Pengusul** | OpenCV Labs, 2011 | D. Lowe, 2004 |
| **Dasar** | FAST + BRIEF | Keypoint scale-invariant + deskriptor |
| **Invarian terhadap** | Orientasi (FAST dimodifikasi) | Skala (dan rotasi) |
| **Matching** | Brute-Force | Brute-Force + test ratio |

# Face Detection: Haar Cascades & Viola-Jones

Cara kerja face detection memakai **Haar Cascades**, komponen kunci framework deteksi objek **Viola-Jones**.

Mendeteksi wajah dengan **cepat** dan mengetahui lokasinya dengan **tepat**; dipakai sebagai tahap awal **Facial Recognition**.

Tahun **2001**: Paul Viola dan Michael Jones mempublikasikan metode berbasis beberapa fitur kunci yang sederhana.

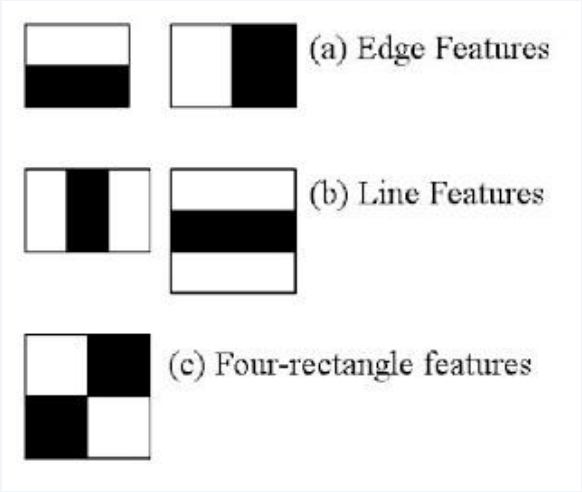
## Tiga ide kunci Viola-Jones

1. **Fitur Haar**
   Fitur sederhana berupa selisih rata-rata area terang dan gelap.

2. **Integral image**
   Dihitung di awal untuk mempercepat waktu komputasi.

3. **Cascade classifier**
   Fitur dihitung secara seri sehingga proses berjalan cepat.

# Fitur Haar & Perhitungan Delta




*Fitur Haar: (a) edge, (b) line, (c) four-rectangle*

| | | | |
|---|---|---|---|
| 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 1 |

| | | | |
|---|---|---|---|
| 0 | 0.1 | 0.8 | 1 |
| 0.3 | 0.1 | 0.7 | 0.8 |
| 0.1 | 0.2 | 0.8 | 0.8 |
| 0.2 | 0.2 | 0.8 | 0.8 |

Tiap fitur adalah **satu nilai**: selisih rata-rata piksel pada area terang dan area gelap.

**Delta = mean(area terang) – mean(area gelap)**

**Area terang**
Sum([0.8, 1, 0.7, 0.8, 0.8, 0.8, 0.8, 0.8]) = 6.5
**Mean = 6.5 / 8 = 0.8125**

**Area gelap**
Sum([0, 0.1, 0.3, 0.1, 0.1, 0.2, 0.2, 0.2]) = 1.2
**Mean = 1.2 / 8 = 0.15**

Delta
**0.6625**
0.8125 – 0.15

Delta di-**threshold** sesuai kebutuhan. Makin tinggi (→ 1), makin cocok sebagai **edge**. Dengan threshold 0,5 dan delta 0,6625, fitur dianggap sebagai **edge**.

# Integral Image & Cascade Classifier

## Integral image

Menghitung delta untuk **seluruh citra** sangat mahal secara komputasi. Viola-Jones mengajukan metode **integral image**.

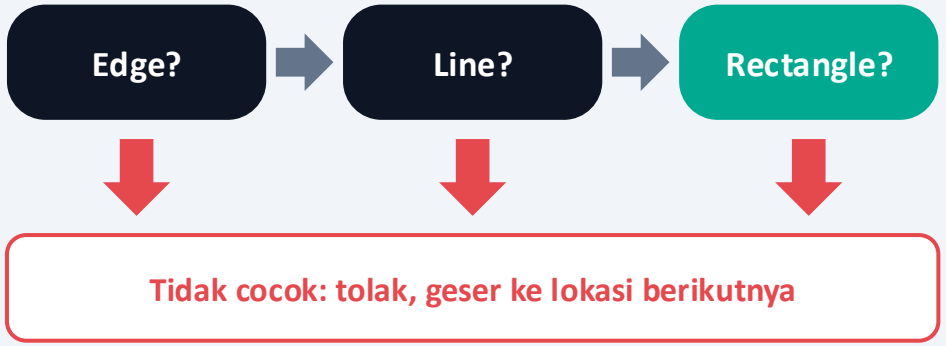
| | | | | | |
|---|---|---|---|---|---|
| 31 | 2 | 4 | 33 | 5 | 36 |
| 12 | 26 | 9 | 10 | 29 | 25 |
| 13 | 17 | 21 | 22 | 20 | 18 |
| 24 | 23 | 15 | 16 | 14 | 19 |
| 30 | 8 | 28 | 27 | 11 | 7 |
| 1 | 35 | 34 | 3 | 32 | 6 |

| | | | | | |
|---|---|---|---|---|---|
| 31 | 33 | 37 | 70 | 75 | 111 |
| 43 | 71 | 84 | 127 | 161 | 222 |
| 56 | 101 | 135 | 200 | 254 | 333 |
| 80 | 148 | 197 | 278 | 346 | 444 |
| 110 | 186 | 263 | 371 | 450 | 555 |
| 111 | 222 | 333 | 444 | 555 | 666 |

15 + 16 + 14 + 28 + 27 + 11 =
101 + 450 – 254 – 186 = 111

## Cascade classifier

Menghitung nilai fitur **secara seri**. Bila fitur edge tidak ditemukan, fitur line dan rectangle **tidak dihitung** dan jendela bergeser ke pencarian berikutnya.




**Lolos semua tahap → kandidat wajah**

# Langkah Deteksi Wajah dengan Cascading

1. Ubah citra masukan ke **grayscale**.
2. Cari fitur awal: **edge feature** yang menunjukkan adanya **mata dan pipi**.
3. Jika fitur tidak ada, dianggap **tidak ada wajah** pada citra tersebut.
4. Jika terindikasi ada, lanjut mencari fitur berikutnya: **nose bridge** (tengah hidung), **lips** (bibir), alis. Pencarian terus dilakukan dengan cascading sehingga terkumpul **ribuan fitur**.



*Citra berwarna → grayscale → wajah terdeteksi (kotak hijau)*

# Penerapan Lain, Kelemahan & Fitur XML

### Dapat dipakai untuk objek lain

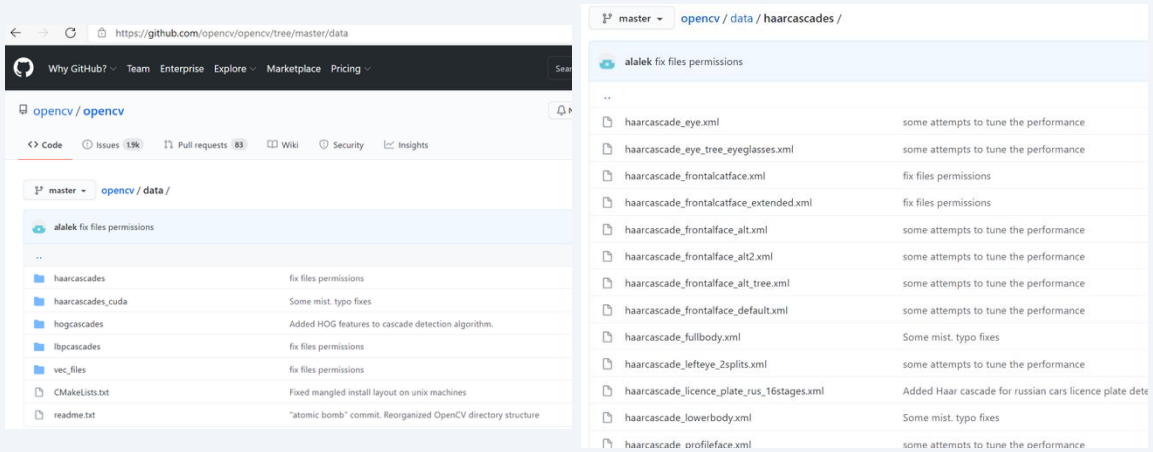
Secara teori berlaku untuk berbagai objek: mata, bibir, hidung; atau mobil, motor, pesawat, dan lainnya.

### Kelemahan

Membutuhkan referensi dataset yang besar untuk mendapatkan fitur yang diinginkan.

### Fitur siap pakai di OpenCV

OpenCV menyediakan fitur yang sering dipakai banyak peneliti, dalam format XML.



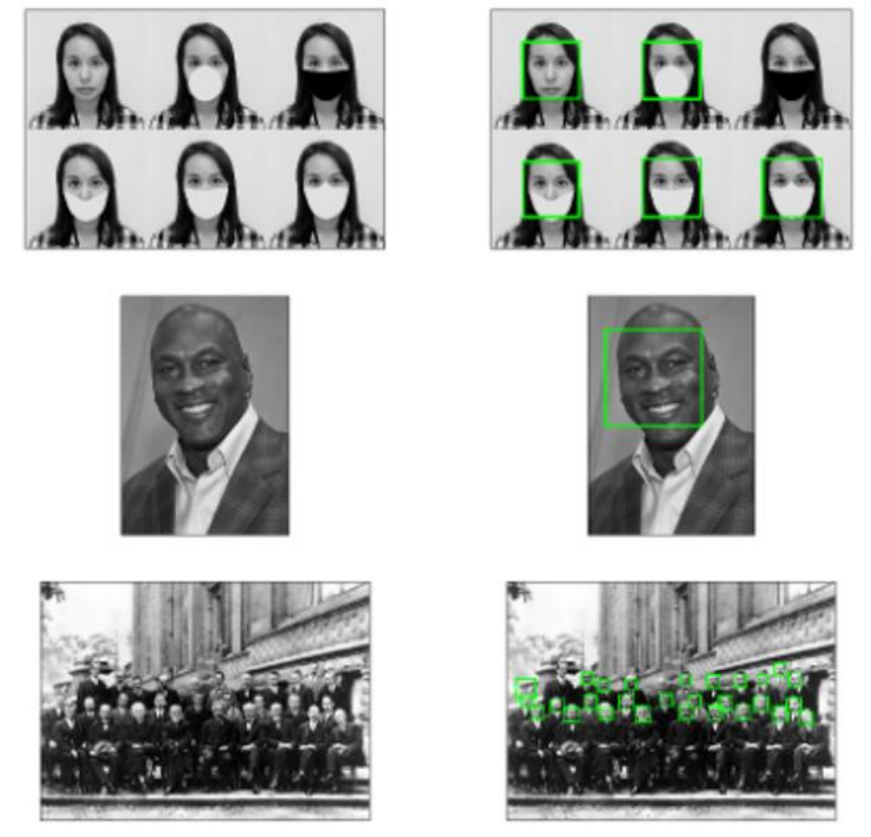

*Berkas XML fitur pada repositori OpenCV (GitHub)*

Contoh penggunaan berkas fitur XML bawaan OpenCV untuk mendeteksi wajah, mata, dan objek lain tanpa melatih dari nol.

# Contoh Hasil Face Detection



*Contoh (1): kiri citra asli, kanan hasil deteksi (kotak hijau = wajah)*



*Contoh (2): kiri citra asli, kanan hasil deteksi (kotak hijau = wajah)*

# TERIMA KASIH!

Pengolahan Citra dan Visi Komputer · RTI245007